

Outline

- Operators
- Storage Class

Operators – Assignment Operator

- An assignment operator is used for assigning a value to a variable. The most common assignment operator is =

Operator	Example	Same as
=	a = b	a = b
+=	a += b	a = a+b
-=	a -= b	a = a-b
*=	a *= b	a = a*b
/=	a /= b	a = a/b
%=	a %= b	a = a%b

Operators – Increment and Decrement

Operator

- C programming has two operators increment ++ and decrement -- to change the value of an operand (constant or variable) by 1.
- Increment ++ increases the value by 1 whereas decrement -- decreases the value by 1.

Operators – Increment and Decrement Operator

Rules for Increment (++) and Decrement (--) operator :

- These two operators are unary operators, meaning they only operate on a single operand.
- If you use the ++ operator as a prefix like: ++var, the value of var is incremented by 1; then it returns the value.
- If you use the ++ operator as a postfix like: var++, the original value of var is returned first; then var is incremented by 1.
- The -- operator works in a similar way to the ++ operator except -- decreases the value by 1.

Operators – Increment and Decrement

Operator cont..

Example :

```
m = 5;
```

```
y = ++m;
```

Output :

```
m = 5
```

```
y = 6
```

```
m = 5;
```

```
y = m++;
```

Output :

```
m = 5
```

```
y = 5
```

Operators – Conditional Operators

- The conditional operator is also known as a **ternary operator**.
- The conditional statements are the decision-making statements which depends upon the output of the expression.
- It is represented by two symbols, i.e., '?' and '!'.
`expression1 ? expression2 : expression3`
- As conditional operator works on three operands, so it is also known as the ternary operator.
- **Syntax :** Expression1? expression2: expression3;

Operators – Conditional Operators cont..

Meaning of the above syntax.

- In the above syntax, the expression1 is a Boolean condition that can be either true or false value.
- If the expression1 results into a true value, then the expression2 will execute.
- The expression2 is said to be true only when it returns a non-zero value.
- If the expression1 returns false value then the expression3 will execute.
- The expression3 is said to be false only when it returns zero value.

```
#include <stdio.h>
int main()
{
    int a=5,b; // variable declaration
    b=((a==5)?(3):(2)); // conditional operator
    printf("The value of 'b' variable is : %d",b);
    return 0;
}
```

Operators – Bitwise Operators

- During computation, mathematical operations like: addition, subtraction, multiplication, division, etc are converted to bit-level which makes processing faster and saves power.
- Bitwise operators are used in C programming to perform bit-level operations.

Operators	Meaning of operators
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
~	Bitwise complement
<<	Shift left
>>	Shift right

Operators – Bitwise Operators

Let us suppose the bitwise AND operation of two integers **12** and **25**.

```
12 = 00001100 (In Binary)
25 = 00011001 (In Binary)

Bit Operation of 12 and 25
  00001100
& 00011001
  -----
  00001000 = 8 (In decimal)
```

Example 1: Bitwise AND

```
#include <stdio.h>

int main() {
    int a = 12, b = 25;
    printf("Output = %d", a & b);

    return 0;
}
```

Operators – Special Operators

- Apart from the above operators, there are some other operators available in C used to perform some specific tasks.
- Some of them are discussed here:
 - sizeof operators
 - comma operators

Operators – Special Operators(sizeof)

- The sizeof is a unary operator that returns the size of data (constants, variables, array, structure, etc)

```
#include <stdio.h>
int main()
{
    int a;
    float b;
    double c;
    char d;
    printf("Size of int=%lu bytes\n",sizeof(a));
    printf("Size of float=%lu bytes\n",sizeof(b));
    printf("Size of double=%lu bytes\n",sizeof(c));
    printf("Size of char=%lu byte\n",sizeof(d));

    return 0;
}
```

Operators – Special Operators(comma)

- Comma operators are used to link related expressions together.
- **For example:** `int a, c = 5, d;`

Precedence of Operators

Precedence of operators come into picture when in an expression we need to decide which operator will be evaluated first. **Operator with higher precedence will be evaluated first.**

Example:

$2 + 3 * 5$

Precedence of multiplication is greater than addition.

$$2 + (3 * 5) = 17$$



$$(2 + 3) * 5 = 25$$



Associativity of Operators

- Associativity in C determines the order of evaluation when two or more operators of the same precedence appear in an expression.
- Operators in C can be either left-to-right or right-to-left associative.

Precedence	Operator	Associativity
1	() [] -> .	Left to right
2	++ --	Left to right
3	++ -- + - !~ (type) * & sizeof	Right to left
4	* / %	Left to right
5	+ -	Left to right
6	<< >>	Left to right
7	<<= >>=	Left to right
8	== !=	Left to right
9	&	Left to right
10	^	Left to right
11		Left to right
12	&&	Left to right
13		Left to right
14	?:	Right to left
15	= + = - = * = / = %= <<= >>= &= ^= =	Right to left

Example

```
#include <stdio.h>

int main() {
    int a = 20, b = 10, c = 5;
    int result = a - b - c; // Left-to-right associativity for '-'

    printf("Result: %d\n", result);
    return 0;
}
```

Explanation:

- Since `-` is left-to-right associative:
 - `(a - b)` is evaluated first: `20 - 10 = 10`.
 - Then, `10 - c`: `10 - 5 = 5`.

Example: Mixed Operators

```
c

#include <stdio.h>

int main() {
    int a = 5, b = 10, c = 15;
    int result = a + b * c / b - a;

    printf("Result: %d\n", result);
    return 0;
}
```

Explanation:

1. Precedence comes first:

- `*` and `/` are evaluated before `+` and `-`.
- `b * c = 10 * 15 = 150`.
- `150 / b = 150 / 10 = 15`.

2. Associativity:

- For `+` and `-` (left-to-right):
 - `a + 15 = 5 + 15 = 20`.
 - `20 - a = 20 - 5 = 15`.

Arithmetic Operations

Program to calculate the sum, difference, product, quotient, and remainder of two numbers.

```
c

#include <stdio.h>
int main() {
    int a = 10, b = 3;
    printf("Sum: %d\n", a + b);
    printf("Difference: %d\n", a - b);
    printf("Product: %d\n", a * b);
    printf("Quotient: %d\n", a / b);
    printf("Remainder: %d\n", a % b);
    return 0;
}
```


Area of a Circle

Program to calculate the area of a circle using the formula $\text{area} = \pi * r * r$.

```
c

#include <stdio.h>
int main() {
    float radius = 5.5;
    float area = 3.14 * radius * radius;
    printf("Area of the circle: %.2f\n", area);
    return 0;
}
```

Average of Three Numbers

Program to find the average of three numbers.

c

```
#include <stdio.h>
int main() {
    int a = 10, b = 20, c = 30;
    float average = (a + b + c) / 3.0;
    printf("Average: %.2f\n", average);
    return 0;
}
```


Swap Two Variables

Program to swap two variables using a temporary variable.

```
c

#include <stdio.h>
int main() {
    int a = 5, b = 10, temp;
    temp = a;
    a = b;
    b = temp;
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

Relational Operations

Program to demonstrate relational operators.

c

```
#include <stdio.h>
int main() {
    int x = 10, y = 20;
    printf("x < y: %d\n", x < y);
    printf("x > y: %d\n", x > y);
    printf("x == y: %d\n", x == y);
    printf("x != y: %d\n", x != y);
    return 0;
}
```


Increment and Decrement

Program to demonstrate prefix and postfix increment and decrement.

C

```
#include <stdio.h>
int main() {
    int a = 5;
    printf("Prefix Increment: %d\n", ++a);
    printf("Postfix Increment: %d\n", a++);
    printf("Postfix Decrement: %d\n", a--);
    printf("Prefix Decrement: %d\n", --a);
    return 0;
}
```

Compound Assignment

Program to demonstrate compound assignment operators.

```
c

#include <stdio.h>
int main() {
    int a = 10;
    a += 5; // a = a + 5
    printf("After += : %d\n", a);
    a *= 2; // a = a * 2
    printf("After *= : %d\n", a);
    return 0;
}
```


Calculate Simple Interest

Program to calculate simple interest.

C

```
#include <stdio.h>

int main() {
    float principal = 1000, rate = 5, time = 2;
    float interest = (principal * rate * time) / 100;
    printf("Simple Interest: %.2f\n", interest);
    return 0;
}
```

Area of a Rectangle

Program to calculate the area of a rectangle.

C

```
#include <stdio.h>
int main() {
    int length = 10, width = 5;
    int area = length * width;
    printf("Area of the rectangle: %d\n", area);
    return 0;
}
```


Volume of a Cube

Program to calculate the volume of a cube.

c

```
#include <stdio.h>
int main() {
    int side = 4;
    int volume = side * side * side;
    printf("Volume of the cube: %d\n", volume);
    return 0;
}
```

Calculate BMI

Program to calculate Body Mass Index (BMI).

C

```
#include <stdio.h>

int main() {
    float weight = 70, height = 1.75; // weight in kg, height in meters
    float bmi = weight / (height * height);
    printf("BMI: %.2f\n", bmi);
    return 0;
}
```


Compound Expressions

Program to evaluate a complex expression.

```
c

#include <stdio.h>
int main() {
    int a = 10, b = 5, c = 3;
    int result = a + b * c - b / a;
    printf("Result: %d\n", result);
    return 0;
}
```

Exercise

1. Average of three numbers.
2. Area of Circle
3. Area of Triangle
4. Ferhenite to Celcius
5. Swap two numbers
6. ASCII value of character
7. Interest calculation
8. Electricity bill calculation

Storage Class – Auto Storage Class

In C, storage classes define the scope, lifetime, and memory location of variables.

auto Storage Class

- ✓ Default storage class for local variables.
- ✓ **Scope:** Local to the block in which they are defined.
- ✓ **Lifetime:** Created when the block is entered and destroyed when the block is exited.
- ✓ **Memory Location:** Stored in the stack.
- ✓ The auto keyword is rarely used explicitly because local variables are auto by default.

Storage Class – Auto Storage Class (Example)

```
#include <stdio.h>
```

```
void function() {  
    auto int x = 10; // 'auto' is optional (local variables are auto  
by default)  
    printf("x = %d\n", x);  
}
```

```
int main() {  
    function();  
    // x is not accessible here  
    return 0;  
}
```

Output : x = 10

Static Storage Class

Scope: Local to the block in which they are defined (like auto).

Lifetime: Preserves the value of the variable between function calls. It is initialized only once and retains its value throughout the program's execution.

Memory Location: Stored in the data segment.

Static Storage Class - Example

```
#include <stdio.h>
```

```
void function() {  
    static int count = 0; // Static variable  
    count++;  
    printf("Count = %d\n", count);  
}
```

```
int main() {  
    function(); // First call  
    function(); // Second call  
    function(); // Third call  
    return 0;  
}
```

Output :

Count = 1
Count = 2
Count = 3

Thank You

